Mathematisch-Naturwissenschaftliche Fakultät

Lernbasierte Computer Vision

Masterarbeit

ScholarCopilot with Fine-Grained Citation Retrieval

Eberhard Karls Universität Tübingen
Mathematisch-Naturwissenschaftliche Fakultät
Wilhelm-Schickard-Institut für Informatik
Lernbasierte Computer Vision
Niklas Munkes, niklas.munkes@student.uni-tuebingen.de, 2025

Bearbeitungszeitraum: 14.04.-14.10.2025

Gutachter: Prof. Dr. Andreas Geiger, Universität Tübingen
Zweitgutachter: Prof. Dr. Carsten Eickhoff, Universität Tübingen
Betreuer: Haoyu He, Universität Tübingen

Selbstständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Masterarbeit selbständig und nur mit den angegebenen Hilfsmitteln angefertigt habe und dass alle Stellen, die dem Wortlaut oder dem Sinne nach anderen Werken entnommen sind, durch Angaben von Quellen als Entlehnung kenntlich gemacht worden sind. Diese Masterarbeit wurde in gleicher oder ähnlicher Form in keinem anderen Studiengang als Prüfungsleistung vorgelegt.

Niklas Munkes (Matrikelnummer 4269436), September 3, 2025

Abstract

TODO

Contents

1 Introduction

TODO

citations build trust [DOW+24]

1.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG, [LPP+21]) models are designed to generate text that is grounded in real-world factual knowlege. They have been widely adopted for knowledge-intensive NLP tasks [SQK+25] such as long-form question answering [XWL+24], scientific question answering [LOS+23] and question synthesis [XLS+24], generation with sentence-level citations [ZBL+24, XWL+24] and academic text generation [AHS+24, WMN+25]. RAG models usually consist of a generator, a pre-trained sequence-to-sequence (seq2seq) model (e.g. a Transformer-based [VSP+17] decoder, see section 2), and a retriever with access to a retrieval corpus, which may be implemented as a dense vector index (e.g. a HNSW [MY16] index, see section 3.2.1) [LPP+21]. Thus they combine pre-trained paramentric memory with non-paramentric (retrieval-based) memory [LPP+21].

The RAG architecture [LPP+21] is comprised of a retriever p_η consisting of a query encoder q and a retrieval corpus d, and a generator p_θ (see Figure 1.1). Given a query x, the retriever $p_\eta(z|x)$ (with parameters η) computes a distribution over the text documents z in the retrieval corpus d according to x [LPP+21]. This distribution is used to obtain the top-k most relevant documents with respect to the query. The generator $p_\theta(y_i|x, z, y_{1:i-1})$, parametrized by θ, then predicts each next token y_i based on the previous tokens $y_{1:i-1}$, the given query x and a document z from the top-k retrieved documents (according to p_η) [LPP+21]. Lewis et al. base their retriever on the `Dense Passage Retriever` [KOM+20] and use a pre-trained `BART-large` [LLG+19] as the generator.

By using an external retrieval corpus, the knowledge RAG models use to inform their generation can directly be inspected and interpreted [LPP+21]. Their TODO

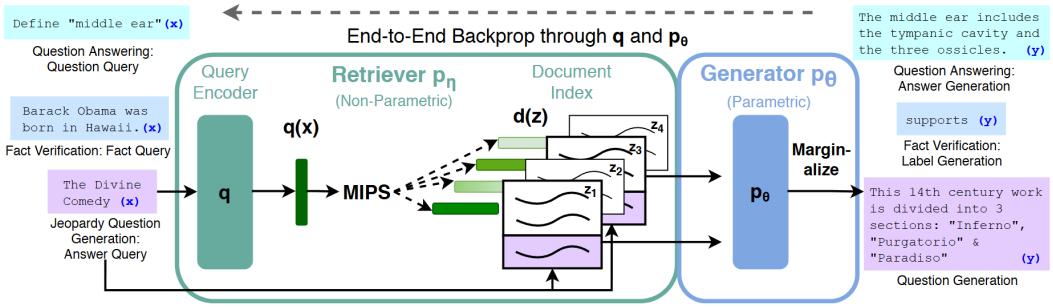

Figure 1.1: Overview of the RAG architecture initially proposed by [LPP+21]. For finding the top-k documents Z', the authors used Maximum Inner Product Search (MIPS).

2 Related Work

TODO

Previous work on text generation with citations has recently transitioned from treating retrieval and generation as seperate processing stages (e.g. [SHC+24]) to alternating between the two (e.g. [XWL+24], [AHS+24]). *Attribute First, then Generate* [SHC+24] initially retrieves a set of relevant sub-sentence spans which are then grouped into coherent clusters which in turn are used to inform an iterative sentence-by-sentence generation step [SHC+24]. *ReClaim* [XWL+24], more specifically the *ReClaim with Interleaving Generation* method, is used to generate an output sequence $O = \{r_1, c_1, ..., r_n, c_n\}$ of alternating references and claims where claim c_i is the portion of the models response that was generated based on reference r_i. Claim and reference generation is performed by seperate LLMs specifically trained for their corresponding task [XWL+24]. In contrast, *OpenScholar* [AHS+24] first retrieves and reranks a set of papers P relevant to the given query. These papers inform a subsequent generation step that produces a generated response r_i [AHS+24]. This response is refined through iterative *self-feedback* where during each iteration of the feedback loop $F = f_1, ..., f_T$, the generator model generates sentences f_j that describe potential improvements to the response and may also prompt the retrieval pipeline to retrieve additional papers, should r_i require more information to generate an improved response r_{i+1} [AHS+24]. Depending on the size of P and the choice of T, OpenScholar still performs retrieval and generation largely seperately.

ScholarCopilot [WMN+25] is a RAG system developed for generating academic-style text with accurate citations. It is intended as a tool that assists the user with academic paper writing by providing iterative text generation with automated citation retrieval [WMN+25]. ScholarCopilot also enables user interaction during generation [WMN+25] by allowing the user to rewrite the generated text or forcefully trigger citation retrieval between the consecutive generation steps but this feature is beyond the scope of this thesis and is thus not utilized for the experiments. ...

TODO

3 Methods

TODO

3.1 Dataset

TODO

3.2 ScholarCopilot

Instead of treating retrieval and generation as separate processing stages (see section 1.1) ScholarCopilot [WMN+25] integrates the information retrieval directly in the generation process. The model is trained to generate *retrieval tokens* (`[RET]`), that upon being generated, cause the model to interupt the generation process to retrieve a reference from the retrieval corpus [WMN+25]. The ScholarCopilot retrieval corpus[1] consists of abstracts of scientific papers, which are encoded by the ScholarCopilot model prior to training/inference and stored as a HNSW [MY16] index using Faiss [DGD+24] for efficient retrieval (see section 3.2.1). Standard ScholarCopilot uses the retrieved abstract to inform its subsequent generation by replacing the `[RET]` tokens in the generated context with the corresponding abstracts.

ScholarCopilot is intended to assist with academic writing by providing "[...] text completion and citation suggestions" (see the official demo[2,3] for an example). In the following we will thus assume that the model is used to generate a scientific paper with citations. Let

$$Y^{\text{in}} = y_1^{\text{in}}, ..., y_l^{\text{in}} \tag{3.1}$$

denote the initial context given to the model. Since we seek to generate a scientific document, this context may consist of the title and abstract of the paper the model is supposed to generate. Let furthermore

$$Y_{1:i-1}^{\text{gen}} = y_1^{\text{gen}}, ..., y_{j-1}^{\text{gen}}, c_j^k, y_{j+1}^{\text{gen}}, ..., y_{i-1}^{\text{gen}} \tag{3.2}$$

[1] `https://huggingface.co/datasets/TIGER-Lab/ScholarCopilot-Data-v1`
[2] `https://huggingface.co/spaces/TIGER-Lab/ScholarCopilot`
[3] `https://www.youtube.com/watch?v=QlY7S52sWDA`

Algorithm 1: TODO

Input: The initial context $Y^{\texttt{in}}$
Output: The generated paper $X_{0:m}$ up to generation step m
Algorithm: getILP(S)

δ_S_ObjectiveFunction $\leftarrow$ **max** $\sum_{i=1}^{n} \sum_{j=1}^{3} x_{ij}$
δ_S_Constraints $\leftarrow \emptyset$
for $i = 1...n$ **do**
  for $j = 1...3$ **do**
    $y_j \leftarrow x_{ij}$
    if $\neg c_{ij}$ **then**
      $y_j \leftarrow x_{ij} + 1 \mod 2$
    end
    δ_S_Constraints $\leftarrow \delta_S$_Constraints $\cup \{y_1 + y_2 + y_3 \geq 1\}$
  end
end
δ_S_Constraints $\leftarrow \delta_S$_Constraints $\cup \{x_{ij} \in \{0,1\}\}$
return δ_S*_ObjectiveFunction*, δ_S*_Constraints*

denote a token sequence generated by the model where c_j^k denotes the k-th `[RET]` token generated by the model. Note that the model may generate multiple retrieval tokens, and that the number of generated retrieval tokens is equal to the number of citations in the generated paper. The representation of the `[RET]` token computed by the model serves as an aggregated representation of the token sequence generated up to the `[RET]` token (e.g. $Y^{\texttt{gen}}_{1:j-1}$ in Eq. 3.2), in the same manner as BERT [DCLT19] (and also for example CF-ViT [CLL+22]) uses the `[cls]` token as a representation of its input sequence. TODO

The ScholarCopilot generator $p_G(y_i|Y^{\texttt{in}}Y^{\texttt{gen}}_{1:i-1})$ predicts the next token y_i $p_R(z|c_j^k)$ denote the ScholarCopilot generator and retriever respectively.

3.2.1 HNSW

TODO

3.2.2 ScholarCopilot Training

TODO

3.3 Passage Reranking

TODO

3.3.1 Batched Self-Consistency

TODO

3.3.2 Min-Max Normalization

TODO

3.4 Deepseek R1

[STB25] TODO

4 Experiments

TODO

4.1 Retrieval Evaluation

TODO

4.2 Generation Evaluation

TODO

4.3 Additional Experiments

TODO

5 Limitations and Future Work

TODO

6 Conclusion

TODO

Bibliography

[AHS+24] Akari Asai, Jacqueline He, Rulin Shao, Weijia Shi, Amanpreet Singh, Joseph Chee Chang, Kyle Lo, Luca Soldaini, Sergey Feldman, Mike D'arcy, David Wadden, Matt Latzke, Minyang Tian, Pan Ji, Shengyan Liu, Hao Tong, Bohao Wu, Yanyu Xiong, Luke Zettlemoyer, Graham Neubig, Dan Weld, Doug Downey, Wen tau Yih, Pang Wei Koh, and Hannaneh Hajishirzi. Openscholar: Synthesizing scientific literature with retrieval-augmented lms, 2024.

[CLL+22] Mengzhao Chen, Mingbao Lin, Ke Li, Yunhang Shen, Yongjian Wu, Fei Chao, and Rongrong Ji. Cf-vit: A general coarse-to-fine method for vision transformer, 2022.

[DCLT19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.

[DGD+24] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. 2024.

[DOW+24] Hyo Jin Do, Rachel Ostrand, Justin D. Weisz, Casey Dugan, Prasanna Sattigeri, Dennis Wei, Keerthiram Murugesan, and Werner Geyer. Facilitating human-llm collaboration through factuality scores and source attributions, 2024.

[KOM+20] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. Dense passage retrieval for open-domain question answering, 2020.

[LLG+19] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension, 2019.

[LOS+23] Jakub Lála, Odhran O'Donoghue, Aleksandar Shtedritski, Sam Cox, Samuel G. Rodriques, and Andrew D. White. Paperqa: Retrieval-augmented generative agent for scientific research, 2023.

[LPP+21] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau

Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021.

[MY16] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs, 2016.

[SHC+24] Aviv Slobodkin, Eran Hirsch, Arie Cattan, Tal Schuster, and Ido Dagan. Attribute first, then generate: Locally-attributable grounded text generation, 2024.

[SQK+25] Rulin Shao, Rui Qiao, Varsha Kishore, Niklas Muennighoff, Xi Victoria Lin, Daniela Rus, Bryan Kian Hsiang Low, Sewon Min, Wen tau Yih, Pang Wei Koh, and Luke Zettlemoyer. Reasonir: Training retrievers for reasoning tasks, 2025.

[STB25] Shubham Sharma, Sneha Tuli, and Narendra Badam. Challenges and applications of large language models: A comparison of gpt and deepseek family of models, 2025.

[VSP+17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2017.

[WMN+25] Yubo Wang, Xueguang Ma, Ping Nie, Huaye Zeng, Zhiheng Lyu, Yuxuan Zhang, Benjamin Schneider, Yi Lu, Xiang Yue, and Wenhu Chen. ScholarCopilot: Training large language models for academic writing with accurate citations, 2025.

[XLS+24] Fangyuan Xu, Kyle Lo, Luca Soldaini, Bailey Kuehl, Eunsol Choi, and David Wadden. Kiwi: A dataset of knowledge-intensive writing instructions for answering research questions, 2024.

[XWL+24] Sirui Xia, Xintao Wang, Jiaqing Liang, Yifei Zhang, Weikang Zhou, Jiaji Deng, Fei Yu, and Yanghua Xiao. Ground every sentence: Improving retrieval-augmented llms with interleaved reference-claim generation, 2024.

[ZBL+24] Jiajie Zhang, Yushi Bai, Xin Lv, Wanjun Gu, Danqing Liu, Minhao Zou, Shulin Cao, Lei Hou, Yuxiao Dong, Ling Feng, and Juanzi Li. Longcite: Enabling llms to generate fine-grained citations in long-context qa, 2024.